

Cheat Sheet Técnica: Tipos de JOIN en SQL

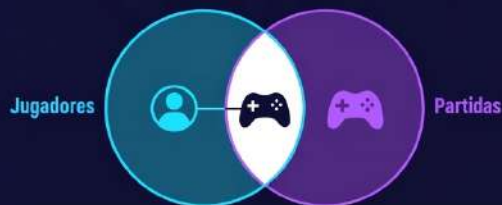
(Edición Matchmaking de Videojuegos)

Referencia técnica visual sobre cómo funcionan las uniones de tablas en SQL utilizando un caso de uso práctico de videojuegos: Jugadores, Partidas y Clanes.

INNER JOIN: La Intersección Exacta

Devuelve solo los registros donde hay una coincidencia en ambas tablas.

Gamer Use Case: Jugadores que están emparejados exactamente en una partida activa. Si un jugador no tiene partida o una partida no tiene jugadores, no aparecen.



```
SELECT * FROM Jugadores J
INNER JOIN Partidas P ON J.partida_id = P.id;
```

LEFT JOIN: Preservar la Lista de Jugadores

Mantiene todos los registros de la tabla izquierda y añade los de la derecha donde hay coincidencia, si no la hay, rellena con NULL.

Gamer Use Case: Todos los jugadores registrados, mostrando sus partidas si están jugando, o NULL si no han iniciado sesión en una partida.



```
SELECT * FROM Jugadores J
LEFT JOIN Partidas P ON J.partida_id = P.id;
```

RIGHT JOIN: Auditoría de Servidores

Mantiene todos los registros de la tabla derecha y añade los de la izquierda donde hay coincidencia.

Gamer Use Case: Todas las partidas activas en el servidor, mostrando sus jugadores. Si una partida está en espera sin jugadores aún, los campos del jugador serán NULL.

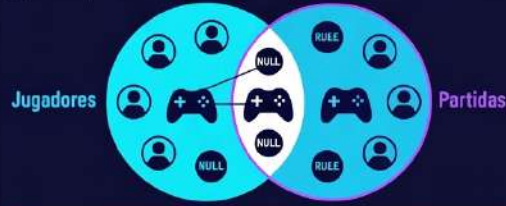


```
SELECT * FROM Jugadores J
RIGHT JOIN Partidas P ON J.partida_id = P.id;
```

FULL OUTER JOIN: Unión Simétrica Total

Devuelve todos los registros cuando hay una coincidencia en cualquiera de las tablas. Nota: En MySQL/SQLite se emula combinando LEFT y RIGHT JOIN con UNION.

Gamer Use Case: Una auditoría completa del servidor: todos los jugadores y todas las partidas, sin importar si están vinculados o no.



```
-- Se emula (MySQL)
SELECT * FROM Jugadores J LEFT JOIN Partidas P ON J.partida_id = P.id
UNION
SELECT * FROM Jugadores J RIGHT JOIN Partidas P ON J.partida_id = P.id;
```

CROSS JOIN: Producto Cartesiano

Crea todas las combinaciones posibles entre las filas de ambas tablas. No requiere una condición ON.

Gamer Use Case: Cruzar todos los jugadores con todos los servidores de región disponibles para generar una matriz de latencia potencial.



```
SELECT J.numero, S.region
FROM Jugadores J
CROSS JOIN Servidores S;
```

SELF JOIN: Jerarquía de Escuadrones

Una tabla se une consigo misma utilizando alias para distinguir las dos instancias.

Gamer Use Case: Mapear a un jugador con su respectivo Líder de Clan dentro de la misma tabla de jugadores.



```
SELECT J.numero AS Niembro, L.numero AS Líder
FROM Jugadores J
JOIN Jugadores L ON J.lider_id = L.id;
```

Tabla de Referencia de Comportamiento

Tipo de JOIN	Registros de Jugadores (A)	Registros de Partidas (B)	Resultado Visual (Conjuntos)
INNER	Solo con pareja	Solo con pareja	Intersección pura ($A \cap B$)
LEFT	Todos	Solo los emparejados	A completo + intersección
RIGHT	Solo los emparejados	Todos	B completo + intersección
FULL OUTER	Todos	Todos	Unión total ($A \cup B$)
CROSS	Combinación total	Combinación total	Rejilla / Matriz ($A \times B$)
SELF	Jugador (Alias 1)	Líder (Alias 2)	Estructura de árbol

Importancia Técnica

Elegir el JOIN correcto garantiza la integridad de los reportes y el rendimiento del emparejamiento. Un JOIN mal aplicado (como un CROSS JOIN accidental) puede saturar los recursos del servidor.